

```
In [4]: import pandas as pd
import numpy as np
from scipy import stats
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
import random
import pickle
import os
import warnings
import json
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

pd.options.mode.chained_assignment = None
warnings.filterwarnings("ignore", category=DeprecationWarning)
filepath = f'/'
```

```
In [6]: merged_data = pd.read_csv(f'{filepath}/mergePh_user_review_business.csv',
```

Data Feature (After the step of extracting PH data from Yelp)

```
In [8]: # 1. restaurant_df
restaurant_df = pd.read_csv(f'{filepath}/philly_restaurants.csv', index_c
```

```
Out[8]: (6204, 9)
```

```
In [174... restaurant_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6204 entries, 0 to 6203
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   business_id     6204 non-null   object
1   name            6204 non-null   object
2   city            6204 non-null   object
3   state           6204 non-null   object
4   stars           6204 non-null   float64
5   review_count    6204 non-null   int64
6   is_open         6204 non-null   int64
7   categories      6204 non-null   object
8   price_level     6204 non-null   int64
dtypes: float64(1), int64(3), object(5)
memory usage: 436.3+ KB
```

```
In [9]: #price is mainly bewteen 1-2 level,median 2
restaurant_df['price_level'].value_counts()
```

```
Out [9]: price_level
2      3093
1      2731
3       338
4        42
Name: count, dtype: int64
```

```
In [10]: col = ['business_id', 'name', 'stars', 'review_count', 'categories', 'price_level']
restaurant_df1 = restaurant_df[col].copy()
restaurant_df1.head(1)
```

```
Out [10]:
```

	business_id	name	stars	review_count	categories	price_level
0	MTSW4McQd7CbVtyjqoe9mw	St Honore Pastries	4.0	80	Restaurants, Food, Bubble Tea, Coffee & Tea, B...	

```
In [11]: #2. Review data
review_df = pd.read_csv(f'{filepath}/all_philly_reviews.csv')
review_df.head(1)
```

```
Out [11]:
```

	user_id	business_id	stars	text	date
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o- D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	2015-01- 04 00:01:03

```
In [12]: review_user_avg = review_df['stars'].mean(skipna=True) #~3.8
```

```
In [13]: review_df1 = review_df[review_df['stars'] > review_user_avg].copy()
```

```
In [14]: #tips data without [star]
review_df2 = review_df[review_df['stars'].isna()]
```

```
In [15]: from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()

def get_sentiment(text):
    scores = analyzer.polarity_scores(str(text))
    return scores['compound']

review_df2['sentiment'] = review_df2['text'].apply(get_sentiment)
```

```
In [16]: review_df3 = review_df2[review_df2['sentiment'] >= 0].copy()
review_df3.head(2)
```

Out [16]:

	user_id	business_id	stars	text	d
730509	-copOvldyKh1qr-vzkDEvw	MYoRNLb5chwjQe3c_k37Gg	NaN	It's open even when you think it isn't	2008-00:56
730510	FQ-zmWPEG_pjSQx6pt3Efw	3ZynJ94VpIdDlaArmEp2Rg	NaN	Yes, I'm eating here again. Breakfast!	2012-15:16

In [17]: `print(f'There are {review_df.shape[0]} reviews in total. For reviews with`
 There are 826272 reviews in total. For reviews with "stars" data. 500470 of them are above average restaurant stars. And for reviews without "stars" data, there are 85513(out of 95763) are neutural or positive.

In [22]: `#combine review_df1,review_df3`
`col = [ 'user_id', 'business_id', 'stars', 'text', 'date']`
`review_df3 = review_df3[col].copy()`

In [23]: `median_star = review_df['stars'].median(skipna=True)`
`median_star`

Out [23]: 4.0

In [24]: `review_df3['stars'] = review_df3['stars'].fillna(median_star)`
`review_df3.head(1)`

Out [24]:

	user_id	business_id	stars	text	date
730509	-copOvldyKh1qr-vzkDEvw	MYoRNLb5chwjQe3c_k37Gg	4.0	It's open even when you think it isn't	2013-08-18 00:56:08

In [21]: `#review_df3['stars'] = pd.to_numeric(review_df3['stars'], errors='coerce')`
`review_df_clean = pd.concat([review_df1, review_df3], axis=0, ignore_index=True)`

In [467...]: `print(f'There are {review_df_clean.shape[0]} nuetural and positive review`
 There are 585983 nuetural and positive review text in total in Philadelphia.

In [27]: `#3. User data`
`user_df= pd.read_json(f'{filepath}/philly_users_data.json')`
`#user_df.shape#220404`

In [28]: `user_df.head(1)`

Out [28]:

	user_id	name	review_count	yelping_since	useful	funny
0	qVc8ODYU5SZjKXVBgXdI7w	Walker	585	2007-01-25 16:47:26	7217	1259

1 rows x 22 columns

In [29]: *#Drop & Filter "Age">=1 from user and business*

```
#3.1 user's age >= 1 year
user_df['yelping_since'] = pd.to_datetime(user_df['yelping_since'])
user_df['user_age'] = 2022 - user_df['yelping_since'].dt.year
user_df1=user_df[user_df['user_age'] >=1].copy()

review_combine =review_df_clean[review_df_clean['user_id'].isin(user_df1[
review_combine.head(1) #extract "user_id" of users who leave business rev
```

Out [29]:

	user_id	business_id	stars	text	date
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	2015-01-04 00:01:03

In [30]: *#3.2 business age >= 1 year*
#filter the age in review_df

```
review_combine['date'] = pd.to_datetime(review_combine['date'])
review_combine['restaurant_age'] = 2022 - review_combine['date'].dt.year
review_combine2 =review_combine[review_combine['restaurant_age'] >=1].cop
review_combine2.shape

print(f'After controlling the age of >=1 year of users and restaurants, t
```

After controlling the age of >=1 year of users and restaurants, there are 584740 reviews(before it was 585983 reviews).

In [31]:

```
review_combine2.head(1)
cols = ['date', 'restaurant_age']
review_combine3 = review_combine2.drop(columns = cols)
```

In [32]: *# merge data*

```
merged_bsreview = pd.merge(review_combine3, restaurant_df1, on='business_
```

In [33]:

```
merged_bsreview = merged_bsreview.rename(columns = {'review_count':'revie
merged_bsreview.head(1)
```

Out [33]:

	user_id	business_id	stars_user	text_user	name	stars_re
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	Zaika	

```
In [34]: #merged_bsreview.info()
print(f"Found matched {merged_bsreview.shape[0]} valid business records a
```

Found matched 584740 valid business records and 584740 valid user stars

```
In [36]: #merged_bsreview.to_csv(f"{filepath}/output file/mergePh_user_review_busi
#print(f'saved!')
```

Data Clean & Columns Transform

```
In [37]: merged_data.head(1)
```

```
Out[37]:
```

	user_id	business_id	stars_user	text_user	name	stars_re
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	Zaika	

```
In [38]: users = merged_data['user_id'].nunique()
business = merged_data['business_id'].nunique()
print(f"The dataset has {merged_data.shape[0]} business visiting records.
```

The dataset has 584740 business visiting records. 171415 unique users and 6171 unique businesses

```
In [39]: merged_data_cat = merged_data.assign(cat=merged_data['categories'].str.sp
all_cat = list(merged_data_cat['cat'].unique()) #len(all_cat)
```

The "categories" category includes only categories directly related to food and beverage/restaurants; all other non-food-related labels such as beverages, beauty, shopping, entertainment, etc. are removed.

```
In [40]: low_cat = {
    'Colombian': 'Latin American',
    'Bangladeshi': 'Indian',
    'Sardinian': 'Italian',
    'Georgian': 'Middle Eastern',
    'Senegalese': 'African',
    'Hungarian': 'European',
    'Hainan': 'Chinese',
    'Austrian': 'European',
    'Sicilian': 'Italian',
    'Fuzhou': 'Chinese',
    'Teppanyaki': 'Japanese',
    'Japanese Curry': 'Japanese',
    'Izakaya': 'Japanese',
    'Honduran': 'Latin American',
    'Empanadas': 'Fast Food',
    'Food Trucks': 'Fast Food',
    'Food Court': 'Fast Food',
    'Food Stands': 'Fast Food',
    'Street Vendors': 'Fast Food',
    'Fondue': 'European'
}

merged_data_cat['cat'] = merged_data_cat['cat'].replace(low_cat)
```

```
In [41]: food_cat = [
    #Asia
    'Chinese', 'Cantonese', 'Shanghainese', 'Hong Kong Style Cafe', 'Szech
    'Japanese', 'Ramen', 'Sushi Bars',
    'Burmese', 'Cambodian', 'Singaporean', 'Thai',
    'Himalayan/Nepalese', 'Indian', 'Indonesian', 'Korean', 'Laotian',
    'Malaysian', 'Mongolian', 'Pakistani', 'Vietnamese', 'Uzbek',
    'Hot Pot', 'Noodles', 'Pan Asian', 'Asian Fusion',
    # Europe
    'Italian', 'Pasta',
    'Spanish', 'Tapas/Small Plates', 'Belgian', 'British',
    'Cajun/Creole', 'Cuban', 'Dominican', 'French', 'German', 'Greek',
    'Irish', 'Modern European', 'Polish',
    'Portuguese', 'Scandinavian', 'Ukrainian', 'Fish & Chips', 'Creperies',
    #Middle East
    'Ethiopian', 'Halal', 'Israeli', 'Kosher', 'Afghan', 'Arabic',
    'Lebanese', 'Middle Eastern', 'Moroccan', 'Armenian',
    'Turkish', 'Sardinian', 'Falafel', 'Kebab',
    #America
    'American (New)', 'American (Traditional)', 'Mexican', 'Tex-Mex', 'Taco
    'Argentine', 'Brazilian', 'Caribbean', 'Hawaiian',
    'Honduran', 'Latin American', 'Mexican', 'Peruvian', 'Salvadoran',
    'Soul Food', 'Southern', 'Trinidadian',
    'Venezuelan', 'Hot Dogs', 'Cheesesteaks', 'Empanadas',
    #fast food, healthy food, others
    'Barbeque', 'Comfort Food', 'Buffets',
    'Fast Food', 'Pizza', 'Sandwiches', 'Wraps', 'Poutineries', 'Burgers',
    'Acai Bowls', 'Specialty Food', 'Fruits & Veggies', 'Salad', 'Seafood',
    #'Restaurants', 'Food', #'Breakfast & Brunch'
]

merged_data_cat['food_cat'] = np.where(merged_data_cat['cat'].isin(food_cat),
merged_data_cat1 = merged_data_cat[merged_data_cat['food_cat'] == 1]
print(f'There are {len(all_cat)} categories, and {len(food_cat)} of them
```

There are 442 categories, and 99 of them are selected.

```
In [42]: cat_counts = merged_data_cat1['cat'].value_counts()
print(cat_counts.describe())
```

```
count      94.000000
mean      12037.882979
std       20486.372189
min        159.000000
25%       1278.250000
50%       3698.500000
75%      12630.500000
max      119866.000000
Name: count, dtype: float64
```

```
In [43]: cat_dist_top = pd.DataFrame(merged_data_cat1['cat'].value_counts(ascending=False))
cat_dist_tail = pd.DataFrame(merged_data_cat1['cat'].value_counts(ascending=True))
```

```
In [44]: valid_cats = set(food_cat)
merged_data_cat2 = merged_data_cat1[merged_data_cat1['cat'].isin(valid_cats)]
```

```
In [45]: users2 = merged_data_cat2['user_id'].nunique()
business2 = merged_data_cat2['business_id'].nunique()
print(f'Before cleaning the category, it was {business2} unique business
```

Before cleaning the category, it was 6171 unique businesses, 171415 unique users and 3416983 business visiting records. After trimming, the dataset has 1131561 business visiting records. 159053 unique users and 4572 unique businesses.

```
In [46]: merged_data_cat2['cat_count_restaurant'] = merged_data_cat2.groupby('business_id')['cat_count_restaurant'].transform('count')
merged_data_cat2['cat_count_user'] = merged_data_cat2.groupby('user_id')['cat_count_user'].transform('count')
merged_data_cat2.head(2)
```

```
Out [46]:
```

	user_id	business_id	stars_user	text_user	name	stars_restaurant
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	Zaika	
0	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.0	Wow! Yummy, different, delicious. Our favo...	Zaika	

```
In [64]: # fold flat data
merged_data_flat = merged_data_cat2.groupby(['user_id', 'business_id']).as_matrix(
    ['stars_user', 'text_user', 'name', 'stars_restaurant', 'review.count_restaurant', 'price_level', 'cat'],
    lambda x: list(x),
    ['cat_count_restaurant', 'cat_count_user'],
).reset_index()

merged_data_flat.head(2)
```

```
Out [64]:
```

	user_id	business_id	stars_user	text_user	name	stars_restaurant
0	r61b7EpVPkb4UVme5tA	5UN1B7XqZohGuULLNIWL1A	5.00	First time visit and not disappointed. From th...	Pa Bist al Ja Ca	
1	r61b7EpVPkb4UVme5tA	9PZxjhTIU7OgPlzuGi89Ew	4.00	Stopped in for lunch. Nice spot! Will definite...	V	

```
In [68]: merged_data_flat.shape #464080
```

```
Out [68]: (464080, 12)
```

```
In [70]: merged_data_flat['user_visit_count'] = merged_data_flat.groupby('user_id')['visit_count'].transform('count')
```

```
merged_data_flat.head(2)
```

```
Out [70]:
```

	user_id	business_id	stars_user	text_user	nan
0	r61b7EpVPkb4UVme5tA	5UN1B7XqZohGuULLNIWL1A	5.00	First time visit and not disappointed. From th...	Pa Bist al Ja Cæ
1	r61b7EpVPkb4UVme5tA	9PZxjhTIU7OgPIzuGi89Ew	4.00	Stopped in for lunch. Nice spot! Will definite...	V

```
In [71]: # filter businesses reviews & user visits
merged_data_clean = merged_data_flat[(merged_data_flat['user_visit_count']
].reset_index(drop=True)
#merged_data_clean.describe()
```

```
In [72]: users_clean = merged_data_clean['user_id'].nunique()
business_clean = merged_data_clean['business_id'].nunique()
print(f"Before selecting visit counts, there are {merged_data_cat1.shape[0]} valid records of business(>3 reviews) and user reviews (>5 visits)")
```

Before selecting visit counts, there are 1131561 business visiting record s, 25034 unique users.After cleanning, there are 280112 business visiting records.

```
In [27]: merged_data_clean.to_csv(f"{filepath}merged_ph_dataclean0319.csv", index=False)
print(f"{merged_data_clean.shape[0]} valid records of business(>3 reviews) and user reviews (>5 visits)")
```

```
In [75]: #note! CSV only good for storing "text" , need to turn text back to list
merged_data_clean = pd.read_csv(f"{filepath}/merged_ph_dataclean0319.csv")
merged_data_clean['cat'] = merged_data_clean['cat'].apply(ast.literal_eval)
```

```
-----
-
NameError                                Traceback (most recent call last)
Cell In[75], line 3
      1 #note! CSV only good for storing "text" , need to turn text back to list
      2 merged_data_clean = pd.read_csv(f"{filepath}/merged_ph_dataclean0319.csv")
----> 3 merged_data_clean['cat'] = merged_data_clean['cat'].apply(ast.literal_eval) #to list!
      4 merged_data_clean.head(1)

NameError: name 'ast' is not defined
```

```
In [76]: pd.set_option('display.float_format', '{:.2f}'.format)
merged_data_clean.describe()
```


Out [76]:

	stars_user	stars_restaurant	review.count_restaurant	price_level	cat_coun
count	280112.00	280112.00	280112.00	280112.00	
mean	4.46	3.91	644.00	1.95	
std	0.50	0.48	848.76	0.61	
min	4.00	1.00	6.00	1.00	
25%	4.00	3.50	157.00	2.00	
50%	4.00	4.00	364.00	2.00	
75%	5.00	4.00	748.00	2.00	
max	5.00	5.00	5721.00	4.00	

User-level Data

In [77]: `merged_data_clean.shape`

Out [77]: (280112, 11)

In [78]: `#merged_data_clean.info()#280112`
`#restaurant type they like to visit`
`user_stats = (`
 `merged_data_clean.groupby('user_id')`
 `.agg(`
 `restaurant_star_median=('stars_restaurant', 'median'),`
 `review_count_median=('review.count_restaurant', 'median'),`
 `price_level_median=('price_level', 'median'),`
 `#price_level_mean=('price_level', 'mean'),`
 `restaurant_cat_median = ('cat_count_restaurant', 'median'),`
 `user_star_median = ('stars_user', 'median'),`
 `user_cat_count=('cat_count_user','first'),`
 `user_visit_count= ('user_visit_count', 'first')`
 `).reset_index()`
`)`
`user_stats.shape`

Out [78]: (25034, 8)

In [79]: `#user_stats.describe()`
`user_stats.head(2)`

Out [79]:

	user_id	restaurant_star_median	review_count_median	price_le
0	-4AjktZiHowEIBCMd4CZA	4.00	377.00	
1	ccVMj2PN6Z9qtdOdlung	4.00	247.00	

In [81]: `#user_stats.to_csv(f'{filepath}/user_stats(clean_cat)0319.csv',index=False`
`## Add Friends Data`
`user_stat = pd.read_csv(f'{filepath}/user_stats(clean_cat)0319.csv', inde`
`user_stat.head(1)`

```
In [45]: user_stat.shape
```

```
Out[45]: (25034, 8)
```

```
In [85]: all_friends = user_df[['user_id','friends']]
all_friends1=all_friends[all_friends['user_id'].isin(user_stat['user_id'])]
```

```
In [89]: user_stat2 =pd.merge(user_stat,all_friends, on = 'user_id')
user_stat2.shape
user_stat2.head(2)
```

```
Out[89]:
```

	user_id	restaurant_star_median	review_count_median	price_level
0	-4AjktZiHowEIBCMd4CZA	4.00	377.00	
1	ccVMj2PN6Z9qtdOdlung	4.00	247.00	

```
In [91]: #user_stat2_reviews= merged_data_clean.groupby('user_id')['text_user'].apply(lambda x: x['reviews'])
#user_stat2['reviews'] = user_stat2['user_id'].map(user_stat2_reviews)
```

```
In [93]: user_stat2_friends =user_stat2.assign(friends_id=user_stat2['friends'].str.strip())
user_stat2_friends.shape#pairs:2972196
```

```
Out[93]: (2972196, 10)
```

```
In [94]: uni_friend = user_stat2_friends['friends_id'].nunique()
uni_user =user_stat2_friends['user_id'].nunique()

print(f'There are {user_stat2_friends.shape[0]} friend pairs in total. Among them there are {uni_user} unique user_id and {uni_friend} unique friend_id.'
```

There are 2972196 friend pairs in total. Among them there are 25034 unique user_id and 1453228 unique friend_id.

```
In [96]: n_user_friend=user_stat2_friends.groupby('user_id').agg(friend_count=('friends_id','nunique'))
n_user_friend['friend_count'].describe()
```

```
Out[96]: count    25034.00
mean         118.73
std           318.53
min             1.00
25%             2.00
50%            25.00
75%            126.00
max          10072.00
Name: friend_count, dtype: float64
```

```
In [99]: user_stat2['friend_count'] = user_stat2['user_id'].map(n_user_friend['friend_count'])
```

```
In [100... user_friend_list = user_stat2_friends[['user_id','friends_id']]
```

```
In [120... user_stat2.to_csv(f"{filepath}/user_stat0319.csv",index=False, encoding='utf-8')
user_friend_list.to_csv(f"{filepath}/user_friend_list0319.csv",index=False, encoding='utf-8')
```

Business x Attribute Data

```
In [104]: business_stat = merged_data_clean.drop(columns=['user_id', 'stars_user', 'c
business_stat.shape #4387
```

```
Out[104]: (4387, 7)
```

```
In [111]: #dummy 0/1
from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()
cat_dm = mlb.fit_transform(business_stat['cat'])

cat_dum2 = pd.DataFrame(cat_dm, columns=mlb.classes_, index=business_stat[

business_attr = pd.merge(business_stat[['business_id', 'price_level']],
                        cat_dum2,
                        on='business_id',
                        how='left')#.set_index('business_id')

#business_attr.head()
#business_attr.to_csv(f"{filepath}/Business_attribute0319.csv", index=False
```

User x Business Matrix

```
In [29]: #User x Business Matrix
user_business = merged_data_clean[['user_id', 'business_id']]
user_business.shape#212490->280112
```

```
Out[29]: (280112, 2)
```

```
In [30]: freq_user_business = user_business.groupby(['user_id', 'business_id']).si
freq_user_business.head()
users = freq_user_business['user_id'].nunique() #93670
business = freq_user_business['business_id'].nunique() #5037
print(f"{users} unique users and {business} unique businesses.The total v
```

25034 unique users and 4387 unique businesses.The total visting records fr
om users is 280112.

```
In [31]: # user_business mtx
from scipy import sparse

uni_user = freq_user_business['user_id'].unique()
uni_business = freq_user_business['business_id'].unique()

#map
user_to_idx = {user: idx for idx, user in enumerate(uni_user)}
business_to_idx = {business: idx for idx, business in enumerate(uni_busin

#mtx
row_idx = freq_user_business['user_id'].map(user_to_idx)
col_idx = freq_user_business['business_id'].map(business_to_idx)
values = freq_user_business['frequency'].values
```

```

user_business_matrix = sparse.csr_matrix(  #?
    (values, (row_idx, col_idx)),
    shape=(len(uni_user), len(uni_business))
)

print(f"User x Business Matrix: {user_business_matrix.shape}")

```

User x Business Matrix: (25034, 4387)

```

In [565... #Save documents for R

from scipy.io import mmwrite
mmwrite(f'{filepath}/User_Business_Matrix0319.mtx', user_business_matrix)
#column names, business id
pd.Series(uni_user).to_csv(f'{filepath}/row_user.csv', index=False, header=False)
pd.Series(uni_business).to_csv(f'{filepath}/col_business.csv', index=False, header=False)
#sparse.save_npz(f'{filepath}/User_Business_Matrix0319.npz', user_business_matrix)
print(f'save')

```

save

User Friends List

```

In [122... #omnivores' friends number
omni_friend = pd.read_csv(f'{filepath}/omnivore_friend_list.csv', index_col=0)

omni_friend_id = omni_friend['friends_id'].str.strip().unique()
print(f"There are {len(omni_friend_id)} unique friends('friend_id')")

```

There are 369879 unique friends('friend_id')

```

In [123... merged_data_flat.shape #457842
users1 = merged_data_flat['user_id'].nunique()
business1 = merged_data_flat['business_id'].nunique()
print(f"Before controlling visit counts and review counts, there are {users1} users and {business1} businesses.")

```

Before controlling visit counts and review counts, there are 159053 unique users and 4572 unique businesses.

```

In [125... # the final business list for LDA (extracted from R)
omni_business_list = pd.read_csv(f'{filepath}/filtered_business_list.csv')
merged_data_clean1 = merged_data_flat[merged_data_flat['business_id'].isin(omni_business_list)]
merged_data_clean2 = merged_data_clean1[(merged_data_clean1['user_visit_count'] > 0)]
#merged_data_clean2.shape #458236

```

```

In [126... users2 = merged_data_clean2['user_id'].nunique()
business2 = merged_data_clean2['business_id'].nunique()
omni_friend_df2 = merged_data_clean2[merged_data_clean2['user_id'].isin(omni_friend_id)]

```

```

In [133... print(f"After cleaning, there are {len(omni_friend_df2)} friend IDs ('user_id') visited the same business as the omnivore group (157358 unique users and 3549 unique businesses). 59.0% omnivores' friends has a matched data in PH user-level data.")

```

After cleaning, there are 218835 friend IDs ('user_id') visited the same business as the omnivore group (157358 unique users and 3549 unique businesses). 59.0% omnivores' friends has a matched data in PH user-level data.

```

In [137... #Omnivore User Friend x Business Matrix
omni_friend_df2 = omni_friend_df2.rename(columns={'user_id': 'friend_user_id'})
omni_friend_business = omni_friend_df2[['friend_user_id', 'business_id']]

```

```
freq_omni_friend_business = omni_friend_business.groupby(['friend_user_id'
```

```
In [138... freq_omni_friend_business.shape
```

```
Out[138... (218835, 3)
```

```
In [136... freq_omni_friend_business.to_csv(f'{filepath}/freq_omni_friend_business.c  
print(f'saved as csv')
```

```
saved as csv
```

```
In [ ]:
```

```
In [ ]:
```